Model	WebQS	NQ	TriviaQA
GPT-J	18.5	12.8	43.9
GPT-J + CC	18.4	12.2	45.6
Toolformer (disabled)	18.9	12.6	46.7
Toolformer	**26.3**	**17.7**	**48.8**
OPT (66B)	18.6	11.4	45.7
GPT-3 (175B)	29.0	22.6	65.9

Table 5: Results for various question answering dataset. Using the Wikipedia search tool for most examples, Toolformer clearly outperforms baselines of the same size, but falls short of GPT-3 (175B).

Model	Es	De	Hi	Vi	Zh	Ar
GPT-J	15.2	**16.5**	1.3	8.2	**18.2**	**8.2**
GPT-J + CC	15.7	14.9	0.5	8.3	13.7	4.6
Toolformer (disabled)	19.8	11.9	1.2	10.1	15.0	3.1
Toolformer	**20.6**	13.5	**1.4**	**10.6**	16.8	3.7
OPT (66B)	0.3	0.1	1.1	0.2	0.7	0.1
GPT-3 (175B)	3.4	1.1	0.1	1.7	17.7	0.1
GPT-J (All En)	24.3	27.0	23.9	23.3	23.1	23.6
GPT-3 (All En)	24.7	27.2	26.1	24.9	23.6	24.0

Table 6: Results on MLQA for Spanish (Es), German (De), Hindi (Hi), Vietnamese (Vi), Chinese (Zh) and Arabic (Ar). While using the machine translation tool to translate questions is helpful across all languages, further pretraining on CCNet deteriorates performance; consequently, Toolformer does not consistently outperform GPT-J. The final two rows correspond to models that are given contexts and questions in English.

this would make solving the tasks trivial, especially given that the underlying QA system was finetuned on Natural Questions.

Results are shown in Table 5. Once again, Toolformer clearly outperforms all other models based on GPT-J, this time mostly relying on the Wikipedia search API (99.3%) to find relevant information. However, Toolformer still lags behind the much larger GPT-3 (175B) model. This is likely due to both the simplicity of our search engine (in many cases, it returns results that are clearly not a good match for a given query) and the inability of Toolformer to *interact* with it, e.g., by reformulating its query if results are not helpful or by browsing through multiple of the top results. We believe that adding this functionality is an exciting direction for future work.

4.2.4 Multilingual Question Answering

We evaluate Toolformer and all baseline models on MLQA (Lewis et al., 2019), a multilingual question-answering benchmark. A context paragraph for each question is provided in English, while the question can be in Arabic, German, Spanish, Hindi, Vietnamese, or Simplified Chinese. In order to solve the task, the model needs to be able to understand both the paragraph and the question, so it may benefit from translating the question into English. Our evaluation metric is the percentage of times the model's generation, capped at 10 words, contains the correct answer.

Results are shown in Table 6. Using API calls consistently improves Toolformer's performance for all languages, suggesting that it has learned to make use of the machine translation tool. Depending on the language, this tool is used for 63.8% to 94.9% of all examples; the only exception to this is Hindi, for which the machine translation tool is used in only 7.3% of cases. However, Toolformer does not consistently outperform vanilla GPT-J. This is mainly because for some languages, finetuning on CCNet deteriorates performance; this might be due to a distribution shift compared to GPT-J's original pretraining data.

OPT and GPT-3 perform surprisingly weak across all languages, mostly because they fail to provide an answer in English despite being instructed to do so. A potential reason for GPT-J not suffering from this problem is that it was trained on more multilingual data than both OPT and GPT-3, including the EuroParl corpus (Koehn, 2005; Gao et al., 2020). As an upper bound, we also evaluate GPT-J and GPT-3 on a variant of MLQA where both the context and the question are provided in English. In this setup, GPT-3 performs better than all other models, supporting our hypothesis that its subpar performance on MLQA is due to the multilingual aspect of the task.

4.2.5 Temporal Datasets

To investigate the calendar API's utility, we evaluate all models on TEMPLAMA (Dhingra et al., 2022) and a new dataset that we call DATESET. TEMPLAMA is a dataset built from Wikidata that contains cloze queries about facts that change with time (e.g., "Cristiano Ronaldo plays for ___") as well as the correct answer for the years between 2010 and 2020. DATESET, described in Appendix D, is also generated through a series of templates, but populated using a combination of random dates/durations (e.g., "What day of the week was it 30 days ago?"). Critically, knowing the current date is required to answer these questions.

Model	TEMPLAMA	DATESET
GPT-J	13.7	3.9
GPT-J + CC	12.9	2.9
Toolformer (disabled)	12.7	5.9
Toolformer	**16.3**	**27.3**
OPT (66B)	14.5	1.3
GPT-3 (175B)	15.5	0.8

Table 7: Results for the temporal datasets. Toolformer outperforms all baselines, but does not make use of the calendar tool for TEMPLAMA.

For both tasks, we use the same evaluation as for the original LAMA dataset.

Results shown in Table 7 illustrate that Toolformer outperforms all baselines for both TEMPLAMA and DATESET. However, closer inspection shows that improvements on TEMPLAMA can not be attributed to the calendar tool, which is only used for 0.2% of all examples, but mostly to the Wikipedia search and question answering tools, which Toolformer calls the most. This makes sense given that named entities in TEMPLAMA are often so specific and rare that even knowing the exact date alone would be of little help. The best course of action for this dataset – first querying the calendar API to get the current date, and then querying the question answering system with this date – is not only prohibited by our restriction of using at most one API call per example, but also hard to learn for Toolformer given that all API calls in its training data are sampled independently.

For DATESET, on the other hand, the considerable improvement of Toolformer compared to other models can be fully accredited to the calendar tool, which it makes use of for 54.8% of all examples.

4.3 Language Modeling

In addition to verifying improved performance on various downstream tasks, we also want to ensure that language modeling performance of Toolformer does not degrade through our finetuning with API calls. To this end, we evaluate our models on two language modeling datasets: WikiText (Merity et al., 2017) and a subset of 10,000 randomly selected documents from CCNet (Wenzek et al., 2020) that were not used during training. Perplexities of various models are shown in Table 8. As one would expect, finetuning on CCNet leads to slightly improved performance on a different CCNet subset, but it slightly deteriorates performance on WikiText, presumably because the original pre-

Model	WikiText	CCNet
GPT-J	**9.9**	10.6
GPT-J + CC	10.3	**10.5**
Toolformer (disabled)	10.3	**10.5**

Table 8: Perplexities of different models on WikiText and our validation subset of CCNet. Adding API calls comes without a cost in terms of perplexity for language modeling without any API calls.

training data for GPT-J is more similar to WikiText than our randomly selected subset of CCNet. Most importantly, however, training on $\mathcal{C}^*$ (our dataset annotated with API calls) does not lead to an increase in perplexity compared to training on $\mathcal{C}$ when API calls are disabled at inference time.[8]

4.4 Scaling Laws

We investigate how the ability to ask external tools for help affects performance as we vary the size of our LM. To this end, we apply our approach not just to GPT-J, but also to four smaller models from the GPT-2 family (Radford et al., 2019), with 124M, 355M, 775M and 1.6B parameters, respectively. We do so using only a subset of three tools: the question answering system, the calculator, and the Wikipedia search engine. Apart from this, we follow the experimental setup described in Section 4.1.

Figure 4 shows that the ability to leverage the provided tools only emerges at around 775M parameters: smaller models achieve similar performance both with and without tools. An exception to this is the Wikipedia search engine used mostly for QA benchmarks; we hypothesize that this is because the API is comparably easy to use. While models become better at solving tasks *without* API calls as they grow in size, their ability to make good use of the provided API improves at the same time. As a consequence, there remains a large gap between predictions with and without API calls even for our biggest model.

5 Analysis

Decoding Strategy We investigate the effect of our modified decoding strategy introduced in Section 4.2, where instead of always generating the

[8] We do not evaluate the perplexity of Toolformer with API calls enabled as computing the probability $p_M(x_t \mid x_1, \ldots, x_{t-1})$ of token x_t given $x_1, \ldots, x_{t-1}$ would require marginalizing over all potential API calls that the model could make at position t, which is intractable.

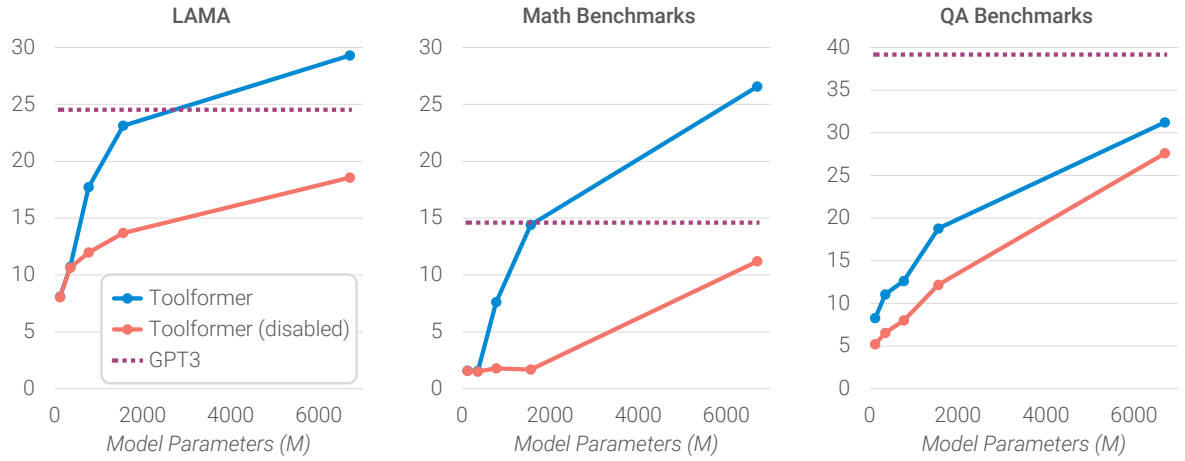

Figure 4: Average performance on LAMA, our math benchmarks and our QA benchmarks for GPT-2 models of different sizes and GPT-J finetuned with our approach, both with and without API calls. While API calls are not helpful to the smallest models, larger models learn how to make good use of them. Even for bigger models, the gap between model predictions with and without API calls remains high.

most likely token, we generate the <API> token if it is one of the k most likely tokens. Table 9 shows performance on the T-REx subset of LAMA and on WebQS for different values of k. As expected, increasing k leads to the model doing API calls for more examples – from 40.3% and 8.5% with $k = 1$ (i.e., regular greedy decoding) to 98.1% and 100% for $k = 10$. While for T-REx, there is already a clear improvement in performance with greedy decoding, on WebQS our model only starts to make a substantial number of API calls as we slightly increase k. Interestingly, for $k = 1$ the model is calibrated to some extent: It decides to call APIs for examples that it would perform particularly badly on without making API calls. This can be seen from the fact that performance on examples where it decides *not* to make an API call (44.3 and 19.9) is higher than average performance if no API calls are made at all (34.9 and 18.9). However, this calibration is lost for higher values of k.

Data Quality We qualitatively analyze some API calls generated with our approach for different APIs. Table 10 shows some examples of texts from CCNet augmented with API calls, as well as the corresponding score $L_i^- - L_i^+$ that is used as a filtering criterion, and whether the API calls made by the model are intuitively useful in the given context. As can be seen, high values of $L_i^- - L_i^+$ typically correspond to useful API calls, whereas low values correspond to API calls that do not provide any information that is useful for predicting future tokens. There are some exceptions, e.g., an API call for "Fast train success" in the fourth example that does not give any relevant information but still reduces perplexity. However, some amount of noise in the API calls that are not filtered can actually be useful as it forces the model finetuned on $\mathcal{C}^*$ to not always blindly follow the results of each call it makes.

	T-REx				WebQS			
k	**All**	**AC**	**NC**	**%**	**All**	**AC**	**NC**	**%**
0	34.9	–	34.9	0.0	18.9	–	18.9	0.0
1	47.8	53.0	44.3	40.3	19.3	17.1	19.9	8.5
3	52.9	58.0	29.0	82.8	**26.3**	26.5	6.6	99.3
10	**53.5**	54.0	22.5	98.1	**26.3**	26.4	–	100.0

Table 9: Toolformer results on the T-REx subset of LAMA and on WebQS for different values of k used during decoding. Numbers shown are overall performance (All), performance on the subset where the model decides to make an API call (AC) and all remaining examples (NC), as well as the percentage of examples for which the model decides to call an API (%).

6 Related Work

Language Model Pretraining There are various approaches that augment language models with some form of additional textual information during pretraining, including various forms of metadata (Keskar et al., 2019), HTML tags (Aghajanyan et al., 2021), Wikipedia markup (Schick et al., 2022), or related texts obtained from an information retrieval system (Guu et al., 2020; Borgeaud et al., 2021; Izacard et al., 2022). For all of these